



PostgreSQL como Motor de Inferência

Tokens, Embeddings e Gradients

João Detomini,

SDE II — EDB / Barman Team

PgConf Brasil 2026

AGENDA

01

A Unidade Fundamental

Tokenização real — como texto vira números

02

O Desafio do Contexto

TOAST, chunking semântico e integridade referencial

03

De Token a Embedding

Os três operadores, espaço vetorial, aritmética semântica

04

RAG Completo Local

Mistral-7B + pgvector + corpus real — sem cloud

05

HNSW e o Otimizador

Grafos de pequenos mundos e ponto de cruzamento

06

Benchmarks Reais

pgvector vs Pinecone — latência e arquitetura

07

Gradientes e ACID

Embedding drift, rollback atômico, fidelidade

08

Encerramento

PostgreSQL como epicentro — RAG + fine-tuning

O que é um Token?

TOKENIZAÇÃO — COMO TEXTO VIRA NÚMEROS

Modelo: all-MiniLM-L6-v2 · WordPiece tokenizer · dados reais

"PostgreSQL" → ['post', '##gre', '##s', '##q', '##l'] ← 5 tokens para 1 palavra

"PostgreSQL é um motor de inferência vetorial."

→
['post', '##gre', '##s', '##q', '##l', 'e', 'um', 'motor', 'de', 'in', '##fer', '##encia', 'veto', '##rial', '.']
7 palavras → 15 tokens (2.14×) · IDs: [101, 2695, 17603, ...] · [CLS]=101 e [SEP]=102 são delimitadores invisíveis

Tipo	Palavras	Tokens	Ratio
Português	10	19	1.90×
Inglês	6	12	2.00×
SQL (SELECT ... <=> ... LIMIT)	11	21	1.91×

"PostgreSQL" → 5 tokens
Modelo nunca viu a palavra inteira

Operador <=> do pgvector
→ ['<', '=', '>'] — 3 tokens

Extensibilidade: novos operadores
sem alterar o core do PostgreSQL

TOAST e Janelas Grandes

TOAST — QUANDO EMBEDDINGS DESAFIAM O STORAGE

PostgreSQL 16 · pgvector 0.8.0 · estratégias reais

O Problema

- Linhas > 8KB → comprimidas e movidas para tabela TOAST
- Embedding 384 dims ≈ 1.5KB por vetor
- Embedding 1536 dims ≈ 6KB — próximo do limite de 8KB
- Updates frequentes → dead tuples → autovacuum crítico
- Estratégia EXTENDED: comprime primeiro, move se necessário

Estratégias Recomendadas

```
-- ✓ Tabela separada para embeddings
CREATE TABLE doc_embeddings (
  doc_id bigint REFERENCES docs(id),
  model_version int DEFAULT 1,
  embedding vector(384)
);
```

Tabela separada HOT update eficiente, VACUUM cirúrgico

model_version Versionar embeddings — rollback sem perda de dados

Chunk semântico Por parágrafo/seção — preserva contexto do LLM

A Física da Representação

OS TRÊS OPERADORES DE SIMILARIDADE

Dados reais · corpus 5.563 chunks · query sobre indexação vetorial



Cosseno

Score medido: 0.5986

Textos de tamanhos variados
Foco na direção, não magnitude
RAG clássico — o mais usado



Produto Escalar

Score medido: 0.5986

Vetores já normalizados
Mais rápido que cosseno
Fine-tuning loops



Euclidiana

Score medido: —

Distância absoluta importa
Clustering semântico
Comparação de magnitude

```
SELECT conteudo, 1 - (embedding <=> $1) AS score  
FROM documentos ORDER BY embedding <=> $1 LIMIT 5;
```

RAG Completo Rodando Local

PIPELINE RAG — TEMPOS REAIS MEDIDOS

MacBook M4 Pro · 48GB · Mistral-7B-Instruct-v0.3-4bit via MLX

PostgreSQL 16.14

pgvector 0.8.0

all-MiniLM-L6-v2

Mistral-7B 4bit

5.563 chunks

Pergunta: "Como o Barman garante a integridade dos backups do PostgreSQL?"

01

Embedding da query

17.8 ms

all-MiniLM-L6-v2
384 dimensões

02

Busca pgvector (HNSW)

1.8 ms ⚡

scores: 0.627, 0.623, 0.616
fonte: concepts.html (Barman)

03

Montagem do contexto

< 1 ms

5 chunks recuperados
passados como prompt

04

Geração Mistral-7B

7.02 s

MLX · Apple Silicon
sem cloud, sem API

O gargalo do RAG é sempre a geração da LLM (~7s) — nunca o banco (1.8ms). pgvector não é o problema de performance: é a solução.

HNSW e o Ponto de Cruzamento

HNSW VS IVFFLAT — QUANDO CADA UM VENCE

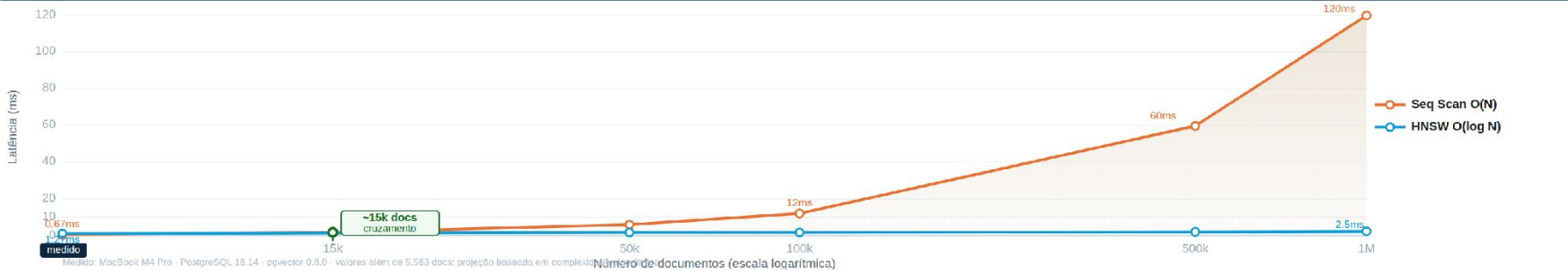
EXPLAIN ANALYZE real · 5.563 documentos · PostgreSQL 16 + pgvector 0.8.0

Característica	IVFFlat	HNSW
Estrutura	Listas invertidas	Grafo hierárquico (small world)
Build time	Rápido	Mais lento
Query speed	Moderado	Muito rápido
Recall	Bom	Excelente
Memória	Baixo	Alto (grafo em RAM)
Melhor para	RAM limitada	Produção, latência crítica

EXPLAIN ANALYZE real (5.563 docs, HNSW):

```
Limit (cost=459..466 rows=5)
  (actual time=0.36..0.40 rows=5)
    -> Index Scan using idx_hnsw
      Order By: embedding <=> '[...]'
Planning Time: 0.570 ms
Execution Time: 0.425 ms
```

Ponto de cruzamento HNSW vs Sequential Scan — $O(\log N)$ vs $O(N)$



Medida: MacBook M4 Pro · PostgreSQL 16.14 · pgvector 0.8.0 · valores além de 5.563 docs: projeção baseada em complexidade algorítmica

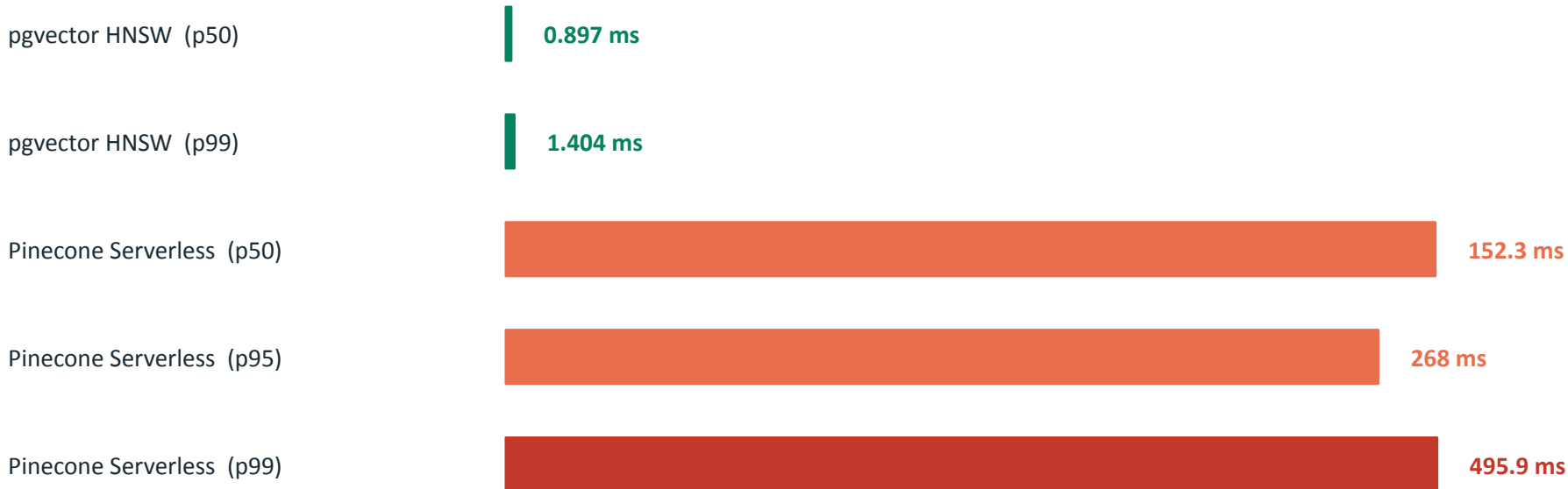


pgvector vs Pinecone

PGVECTOR VS PINECONE — BENCHMARK REAL

5 queries · 30 iterações · MacBook M4 Pro → Pinecone Serverless us-east-1

Latência de query (ms) — Execution Time via EXPLAIN ANALYZE (pgvector) / round-trip medido (Pinecone)



⚠ O Pinecone inclui latência de rede (~156ms HTTP+TLS medidos). pgvector roda localmente — a diferença é arquitetural, não uma limitação do Pinecone.

PGVECTOR VS PINECONE — ALÉM DA LATÊNCIA

Arquitetura, consistência e operação

	pgvector	Pinecone Serverless
Consistência	ACID nativa	Eventual
Latência de rede	Zero (colocado)	~150ms (serviço externo)
Rollback de embeddings	Transacional — WAL	Snapshot externo
Joins relacionais	Mesmo banco	Aplicação
Custo	Self-hosted	Managed / por volume
Portabilidade	PostgreSQL padrão	API proprietária
HNSW	✓ desde v0.5 (2023)	✓ nativo

Por que as big techs usam Pinecone?

Inércia de ecossistema · tooling MLOps integrado · pgvector HNSW chegou em 2023. Quem adotou antes carrega custo de migração — não necessariamente razão técnica.

Onde a IA Encontra a Consistência

EMBEDDING DRIFT — O BANCO QUE MENTE SILENCIOSAMENTE

Experimento real · PostgreSQL 16 · pgvector 0.8.0

Estado 1	Estado 2	Estado 3	Estado 4
			
Estado consistente	Drift silencioso	Correção ACID	Pós-ROLLBACK
Score: 0.4719	Score: 0.4719	Score: 0.4724	Score: 0.4724
Conteúdo e embedding alinhados Resultado correto e relevante	UPDATE no conteúdo SEM regerrar embedding Mesmo score — banco retorna dado errado	BEGIN; UPDATE (conteúdo + embedding); COMMIT Atomicidade garante consistência	Batch corrompido revertido (score -0.03) Estado anterior restaurado

O drift no Estado 2 é silencioso: score permanece 0.4719 com conteúdo errado. Sem garantia transacional, não há como detectar automaticamente.

GRADIENTE SIMULADO + ROLLBACK — FIDELIDADE SOB ACID

$learning_rate=0.15 \cdot direção = \text{vetor da query normalizado}$

v1 Baseline	v2 Fine-tuning	v3 Corrompido	ROLLBACK Restaurado
0.4872	0.5965	-0.0327	0.5965
Embeddings genéricos	Gradiente legítimo +22% de score	Ruído aleatório Inverte semântica	v2 restaurado Fine-tuning preservado

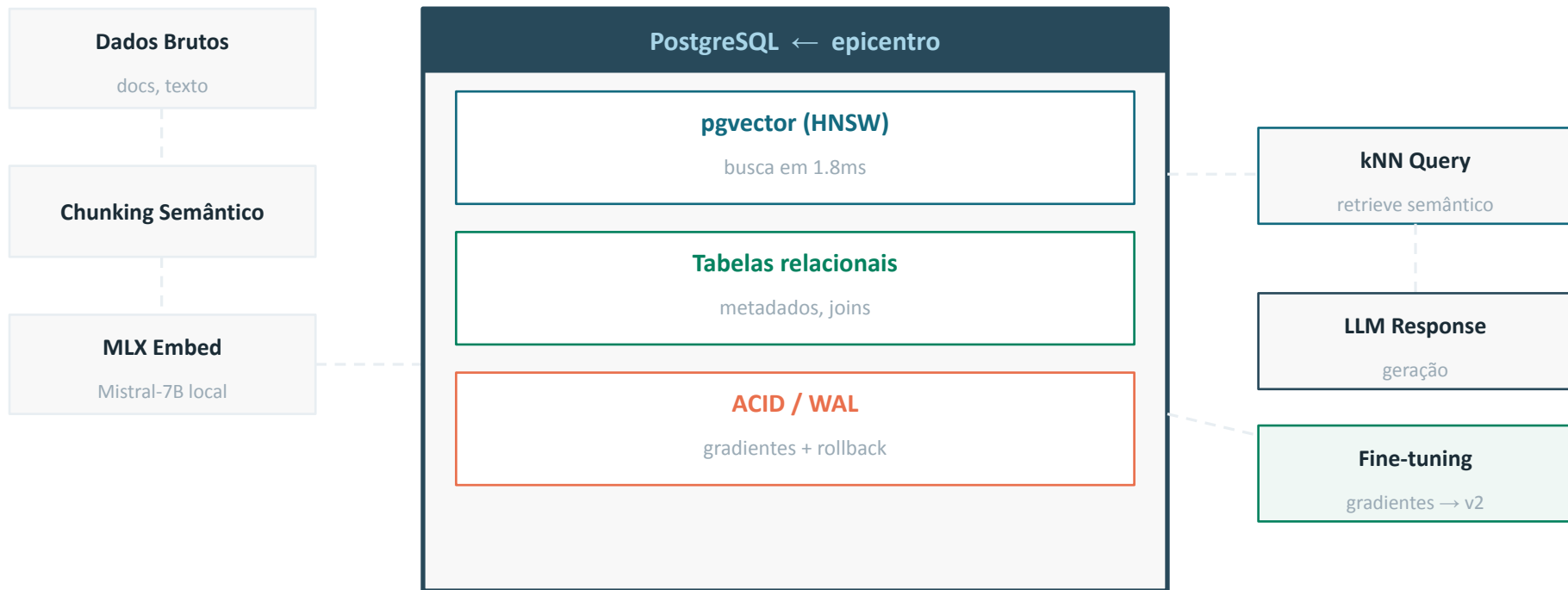
Score -0.0327: ruído aleatório inverte a semântica — degradação detectável antes do commit.

ROLLBACK restaura v2, não v1: o fine-tuning legítimo é preservado. Apenas o batch corrompido é descartado.

PostgreSQL como Epicentro da Inferência

RAG + FINE-TUNING — POSTGRESQL NO CENTRO

Arquitetura completa rodando em hardware de desenvolvedor



NA SEGUNDA-FEIRA DE MANHÃ

Setup mínimo e referências para começar

```
CREATE EXTENSION vector;

CREATE TABLE documentos (
  id bigserial PRIMARY KEY,
  conteudo text,
  model_version int DEFAULT 1,
  embedding vector(384)
);

CREATE INDEX ON documentos
  USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64);

-- Query de similaridade
SELECT conteudo,
  1 - (embedding <=> $1) AS score
FROM documentos
ORDER BY embedding <=> $1
LIMIT 5;
```

Repositórios e leituras

pgvector

github.com/pgvector/pgvector

MLX (Apple Silicon)

github.com/ml-explore/mlx

HNSW Paper (Malkov, 2018)

arxiv.org/abs/1603.09320

PostgreSQL — TOAST docs

postgresql.org/docs/16/storage-toast.html

Barman — pgbarman.org

pgbarman.org/documentation

*“O banco deixa de ser um componente passivo
e se torna parte ativa da inteligência da
aplicação.”*

PostgreSQL como Motor de Inferência: Tokens, Embeddings e Gradients

João Detomini · EDB / Barman Team · PgConf Brasil 2026

